

BIG DATA ANALYTICS ASSIGNMENT

Name: AGASH K

ROLL NO: 727722EUCS009

HADOOP COMPONENTS

1. Pig

Apache Pig is a platform for analysing large datasets that consists of:

- **A high-level data flow language:** Pig Latin
- **An execution framework:** that translates scripts into a series of MapReduce jobs

Pig was developed at Yahoo! to overcome the limitations and complexities of writing raw MapReduce programs. It offers a more intuitive approach to data processing.

Key Features

- Simplifies writing complex MapReduce jobs.
- Handles both **structured and semi-structured** data.
- Supports **ETL (Extract, Transform, Load)** tasks effectively.
- Operates in both **local** and **Hadoop distributed modes**.
- Can extend functionality through **User Defined Functions (UDFs)** in Java, Python, etc.

Pig Architecture

Apache Pig's architecture consists of two main components:

=> Pig Latin Language

Pig Latin is the scripting language used to express data transformations. It is:

- **Procedural** in nature (sequence of steps)
- Easy to learn and write
- Closer to SQL, but designed for parallel dataflow execution

=> Pig Execution Engine

Responsible for executing Pig Latin scripts. It has two modes:

- **Local Mode:** Executes scripts on a single JVM (no Hadoop)
- **MapReduce Mode:** Scripts are converted into MapReduce jobs and executed on Hadoop clusters

Execution Mechanism

Pig converts the Pig Latin script into a **logical plan**, which is optimized and transformed into a **physical plan** consisting of MapReduce jobs.

Execution Flow:

1. **Parsing:** Checks for syntax and schema errors
2. **Optimization:** Logical plan is optimized
3. **Compilation:** Converts optimized plan into a sequence of MapReduce jobs
4. **Execution:** MapReduce jobs are executed on Hadoop

This abstraction allows developers to avoid manual MapReduce job writing.

Advantages of Using Pig

- **Less Code:** 20 lines of Pig Latin could replace 200+ lines of Java MapReduce code
- **Extensible:** UDFs can be written in Java, Python, Ruby
- **Flexible:** Works well with semi-structured data like JSON
- **Efficient:** Optimized execution plans increase performance
- **Readable:** High-level syntax is easier to understand

Limitations of Pig

Despite its usefulness, Pig has some drawbacks:

- Not ideal for real-time processing
- Lacks strict schema enforcement (can lead to runtime errors)
- Less suitable for iterative algorithms (e.g., PageRank)
- Limited debugging tools compared to other frameworks

Conclusion

Apache Pig plays a vital role in simplifying big data processing on Hadoop. Its intuitive scripting language, Pig Latin, abstracts the complexity of MapReduce and allows for rapid development of data transformation logic. While it's not perfect for every scenario—especially real-time or iterative workloads—it remains a powerful tool for batch processing, ETL pipelines, and exploratory analysis in large-scale environments.

2. Hive

Apache Hive is an open-source data warehousing solution for Hadoop that enables:

- Querying and managing large datasets in distributed storage.
- Using **HiveQL (HQL)** – a query language similar to SQL.
- Translating HQL queries into **MapReduce**, **Tez**, or **Spark** jobs for execution.

Features of Hive

- **SQL-like Interface:** Reduces learning curve for SQL users.
- **Schema on Read:** Applies schema only when data is read, not written.
- **Scalable:** Handles petabytes of data.
- **Extensible:** Supports custom User Defined Functions (UDFs).
- **Supports Batch Processing:** Efficient for processing large volumes of historical data.

Hive Architecture

Hive architecture includes various layers and components that help convert HiveQL queries into executable distributed jobs.

Major Components

- **User Interface:** CLI, Hive Web UI, Hive JDBC/ODBC
- **Compiler:** Converts HiveQL into DAG of stages (MapReduce, Tez, or Spark jobs)
- **Execution Engine:** Executes the plan using Hadoop or other engines
- **Metastore:** Stores schema, table metadata, partitions, etc.

Workflow

1. User submits HiveQL query via CLI or Web UI.
2. Compiler converts query to logical plan.
3. Optimizer refines execution plan.
4. Execution engine processes data using Hadoop cluster.
5. Results are returned to the user.

Hive Data Model

Hive organizes data into databases, tables, partitions, and buckets.

Databases and Tables

- **Database:** Logical namespace for tables
- **Table:** Represents structured data (rows and columns)

Partitions

- Used to divide tables into segments based on values (e.g., by date).
- Helps in optimizing query performance.

Buckets

- Further subdivides data inside partitions.
- Useful for sampling, joins, and sorting.

Advantages of Hive

- **Familiar SQL Interface:** Easy for analysts and developers
- **Integration with Hadoop:** Native support for HDFS and YARN
- **Extensible:** Support for custom functions and formats
- **Scalable and Fault-tolerant:** Inherits from Hadoop's scalability

Limitations of Hive

Despite its strengths, Hive also has limitations:

- **Not Real-Time:** Hive is batch-oriented and unsuitable for real-time use cases
- **Latency:** Queries can be slow due to job compilation and execution overhead
- **Limited Support for Updates:** Append-only nature makes it less flexible for transactional systems
- **No Indexing (in earlier versions):** Query optimization could be limited compared to RDBMS

Conclusion

Apache Hive revolutionized big data analytics by bringing the power of SQL to the world of Hadoop. It allowed business users and analysts to process massive amounts of data without needing to write complex MapReduce code. While it's not ideal for real-time queries or transactional systems, Hive continues to be a cornerstone in the batch processing and data warehousing landscape of the Hadoop ecosystem.

3. HBase

Apache HBase is a **NoSQL database** built for applications that require fast and random access to large amounts of data. It provides:

- **Column-oriented** storage
- High throughput for **real-time read/write operations**
- Integration with HDFS for storage and MapReduce for processing

Key Features of HBase

- **Scalability:** Scales horizontally to billions of rows and millions of columns
- **Real-Time Access:** Supports fast random reads and writes
- **Schema-less:** Flexible schema with dynamic column families
- **Fault Tolerant:** Built-in data replication across nodes
- **Tight Hadoop Integration:** Uses HDFS for storage and works with MapReduce, Hive, and Pig

HBase Architecture

Core Components

- **HMaster:** Manages region servers, handles schema changes and load balancing
- **Region Server:** Handles read/write requests, manages regions (horizontal data splits)
- **Zookeeper:** Coordinates between HMaster and Region Servers; ensures high availability
- **HDFS:** Stores the actual data blocks

Data Storage Flow

1. **Client writes** to Region Server
2. Data goes to **MemStore** (in-memory buffer)
3. Periodically flushed to **HFile** in HDFS
4. **WAL (Write Ahead Log)** records every change for data recovery

HBase Data Model

HBase has a very different data model than traditional RDBMS. It is a **sparse, multidimensional, column-oriented** store.

Table Structure

- **Row Key:** Uniquely identifies each row
- **Column Family:** Logical grouping of columns; defined during table creation
- **Column Qualifier:** Name of the column within a family
- **Cell:** Intersection of row and column, stores value and timestamp

Advantages of HBase

- **Real-time Read/Write:** Unlike HDFS/Hive which are batch-oriented
- **Scalable Architecture:** Designed for horizontal scaling
- **Flexible Schema:** Allows sparse, evolving data structures
- **High Availability:** Integration with Zookeeper ensures resilience
- **Fault-Tolerant:** Automatic data recovery via WAL and replication

Limitations of HBase

Despite its power, HBase is not always the best fit.

- **Complex Setup and Tuning:** Requires Zookeeper, memory tuning, and region balancing
- **No SQL Support:** Requires learning a new API or toolset
- **Not ACID Compliant:** Eventual consistency model
- **No Joins or Aggregations:** Must be handled at the application level
- **Large Overhead:** Not efficient for small-scale applications

Conclusion

Apache HBase fills a critical gap in the Hadoop ecosystem by supporting **real-time read and write operations** on large, sparse datasets. It is the go-to solution for applications needing **low latency, flexible schema, and horizontal scalability**. Although it's not a drop-in replacement for traditional RDBMS or suited for transactional systems, HBase is ideal for time-series data, logs, and any use case where speed, volume, and distributed access matter most.

HADOOP ARCHITECTURE ECOSYSTEM

